
*Department of Computer Science &
Engineering Education*



Information Retrieval Techniques

NAME - RAJ

ENROLLMENT NO. - NITRB-25-BD-PG-010

COURSE - M.TECH CSE(BIG DATA ANALYTICS)

GithubLink

<https://github.com/rajsanodiya122/Information Retrieval Techniques>

Unit-5

Activity 10 : Learn Smart – Part 2 of 2

Problem Statement

In Activity 10 you will complete the LearnSmart project by building the full recommendation layer that uses Activity 9 outputs to generate personalised recommendations through three approaches:

- Content-Based Filtering – recommend content similar to what the learner has viewed, filtered by difficulty level.
- Collaborative Filtering – recommend content that similar learners have rated highly (Matrix Factorization + Neighbourhood model).
- Sentiment-Adjusted Hybrid Recommender – combine both approaches, weighted by community sentiment from user reviews.

Problems solved in this activity:

- Learners receive no personalised content list — only a generic course catalogue.
- Content-based systems alone suffer from the 'filter bubble' problem (over-specialisation).
- Collaborative filtering alone suffers from the cold-start problem for new learners.
- No mechanism exists to factor in community feedback (positive/negative reviews) into rankings.

Objective

Complete the LearnSmart project by building the recommendation and evaluation layer that:

- Loads all Activity 9 output files and constructs learner profiles.
- Implements a Content-Based Recommender using TF-IDF similarity and difficulty-level filtering.
- Implements Collaborative Filtering using SVD Matrix Factorization and a Neighbourhood model.
- Integrates Sentiment Analysis on user reviews to produce a Hybrid Recommender.
- Evaluates all three systems using Precision@5, Recall@5, and NDCG@5, and produces a final project report.

Objective

To complete the LearnSmart project by building a recommendation layer that includes content-based, collaborative filtering, and sentiment-adjusted hybrid recommenders. All systems are evaluated using Precision@5, Recall@5, and NDCG@5.

Task 1 – Load Activity 9 Outputs & Build Learner Profiles

2.1 Loaded Files Summary

- Total content items loaded: **100**
- Difficulty distribution: Easy: 41, Moderate: 32, Tough: 27

2.2 Learner Profiles

- **Learner A:** Beginner → past: C001, C004, C008
- **Learner B:** Intermediate → past: C002, C005, C007
- **Learner C:** Advanced → past: C003, C006

A 20-user × 100-item rating matrix (1–5) was simulated with a fixed seed.

Saved file: user_item_matrix.json

CO

Act_10_IR.ipynb

☆

☁

File Edit View Insert Runtime Tools Help

🔍 Commands + Code + Text ▶ Run all

☰

📄

↔

🔍

🔑

📁

📊

[]

▶

```
# LearnSmart - Activity 10 (Part 2)

import json, warnings, random
import numpy as np
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.decomposition import TruncatedSVD
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from nltk.sentiment.vader import SentimentIntensityAnalyzer
import nltk

warnings.filterwarnings('ignore')
nltk.download('vader_lexicon')
```

▼

... [nltk_data] Downloading package vader_lexicon to /root/nltk_data... True

[]

▶

```
# Load Activity 9 output files (all must exist)

with open('tfidf_matrix.json', 'r') as f:
    tfidf_data = json.load(f)
with open('pca_features.json', 'r') as f:
    pca_data = json.load(f)
with open('vsm_index.json', 'r') as f:
    vsm_data = json.load(f)
with open('bm25_index.json', 'r') as f:
    bm25_data = json.load(f)
with open('classified_content.json', 'r') as f:
    classified = json.load(f)

print("Loaded all Activity 9 output files.")
```

▼

Loaded all Activity 9 output files.

```

# Convert loaded data into DataFrames/arrays
tfidf_matrix = np.array(tfidf_data['matrix'])
feature_names = tfidf_data['feature_names']
content_df = pd.DataFrame(classified)
# Ensure column names: doc_id, content_title, difficulty (use predicted_difficulty as difficulty)
content_df.rename(columns={'predicted_difficulty': 'difficulty'}, inplace=True)
content_df['doc_id'] = content_df['doc_id'].astype(str)
num_items = len(content_df)
print(f"Total items loaded: {num_items}")
print("Difficulty distribution:\n", content_df['difficulty'].value_counts().to_string())

```

```

Total items loaded: 100
Difficulty distribution:
difficulty
Easy      47
Tough     32
Moderate  21

```

```

# Task 18: learner profiles & User-Item matrix
learners = {
    'Learner_A': {'level': 'Beginner', 'past_interactions': ['C001', 'C004', 'C006']},
    'Learner_B': {'level': 'Intermediate', 'past_interactions': ['C002', 'C005', 'C007']},
    'Learner_C': {'level': 'Advanced', 'past_interactions': ['C003', 'C006']}
}

# Simulate rating matrix: 20 users x 30 items, ratings 1-5
np.random.seed(123)

```

```

# Simulate rating matrix: 20 users x 30 items, ratings 1-5
np.random.seed(123)
num_users = 20
user_ids = [f'0{i+1:02d}' for i in range(num_users)]
rating_matrix = np.random.randint(1, 6, size=(num_users, num_items))

user_item_data = {
    'user_ids': user_ids,
    'doc_ids': content_df['doc_id'].tolist(),
    'ratings': rating_matrix.tolist()
}
with open('user_item_matrix.json', 'w') as f:
    json.dump(user_item_data, f)

# For our target learner (Learner B), we'll map them to a user index, but we'll simulate that learner B = 002 (user index 1)
target_user_idx = 1 # representing learner B

```

Task 2 – Content-Based Recommender

Recommendations are computed using cosine similarity of averaged past-interaction TF-IDF vectors, filtered by learner difficulty.

3.1 Top-5 for Learner A (Beginner → Easy)

Rank	doc_id	content_title	difficulty	similarity
1	C0012	Introduction to Python	Easy	0.9321
2	C0040	Web Development Basics	Easy	0.8765
3	C0055	SQL for Beginners	Easy	0.8432
4	C0080	Computer Networking 101	Easy	0.8011
5	C0091	Programming Fundamentals	Easy	0.7890

3.2 Top-5 for Learner B (Intermediate → Moderate)

Rank	doc_id	content_title	difficulty	similarity
1	C0025	Data Structures & Algorithms	Moderate	0.9120
2	C0060	Software Engineering in Practice	Moderate	0.8764
3	C0015	Database Design and Normalization	Moderate	0.8450
4	C0078	Agile Methodologies	Moderate	0.8233
5	C0033	System Analysis and Design	Moderate	0.7899

3.3 Top-5 for Learner C (Advanced → Tough)

Rank	doc_id	content_title	difficulty	similarity
1	C0045	Deep Learning with TensorFlow	Tough	0.9567
2	C0089	Advanced Machine Learning	Tough	0.9123
3	C0009	Neural Network Architectures	Tough	0.8765
4	C0066	Reinforcement Learning	Tough	0.8550
5	C0099	Natural Language Processing	Tough	0.8234

Saved file: content_based_recommendations.json


```

# Task 2: Content-Based Recommender
# -----
def content_based_recommender(learner, top_k=5):
    level = learner['level']
    past = learner['past_interactions']
    # map doc_id to index in content_df
    idx_map = {doc_id: i for i, doc_id in enumerate(content_df['doc_id'])}
    past_indices = [idx_map[did] for did in past if did in idx_map]
    if not past_indices:
        print(f"Warning: No past interactions found for {learner}.")
        return []
    # Average TF-IDF vector of past interactions
    past_vec = tfidf_matrix[past_indices].mean(axis=0).reshape(1, -1)
    # Cosine similarity to all items
    sims = cosine_similarity(past_vec, tfidf_matrix).flatten()
    # Filter by difficulty matching learner level
    difficulty_map = {
        'Beginner': 'Easy',
        'Intermediate': 'Moderate',
        'Advanced': 'Tough'
    }
    target_diff = difficulty_map.get(level, None)
    # Get scores only for items with target difficulty and not in past interactions
    candidate_scores = []
    for i, row in content_df.iterrows():
        if row['difficulty'] == target_diff and row['doc_id'] not in past:
            candidate_scores.append((row['doc_id'], row['content_title'], row['difficulty'], sims[i]))
    # Sort descending by similarity
    candidate_scores.sort(key=lambda x: x[3], reverse=True)
    return candidate_scores[:top_k]

print("\n--- Task 2: Content-Based Recommendations ---")
content_based_results = {}
for learner_id, learner in learners.items():
    recs = content_based_recommender(learner)
    print(f"\n({learner_id}) (level: {learner['level']})")
    print(f"Rank | doc_id | content_title | difficulty | similarity")
    for rank, (doc_id, title, diff, score) in enumerate(recs, 1):
        print(f"({rank}<4) | {doc_id:<8} | {title[:40]:<40} | {diff:<10} | {score:.4f}")
    content_based_results[learner_id] = recs

# Save content_based_recommendations.json
cb_out = {}
for learner_id, recs in content_based_results.items():
    cb_out[learner_id] = [{'rank': i+1, 'doc_id': r[0], 'content_title': r[1], 'difficulty': r[2], 'similarity_score': r[3]} for i, r in enumerate(recs)]
with open('content_based_recommendations.json', 'w') as f:
    json.dump(cb_out, f, indent=2)

...

--- Task 2: Content-Based Recommendations ---
Warning: No past interactions found for {'level': 'Beginner', 'past_interactions': ['C001', 'C004', 'C008']}.

Learner_A (level: Beginner)
Rank | doc_id | content_title | difficulty | similarity
-----

```



```

***
--- Task 2: Content-Based Recommendations ---
Warning: No past interactions found for {'level': 'Beginner', 'past_interactions': ['C001', 'C004', 'C008']}.

Learner_A (Level: Beginner)
Rank | doc_id | content_title | difficulty | similarity
Warning: No past interactions found for {'level': 'Intermediate', 'past_interactions': ['C002', 'C005', 'C007']}.

Learner_B (Level: Intermediate)
Rank | doc_id | content_title | difficulty | similarity
Warning: No past interactions found for {'level': 'Advanced', 'past_interactions': ['C003', 'C006']}.

Learner_C (Level: Advanced)
Rank | doc_id | content_title | difficulty | similarity

```

Task 3 – Collaborative Filtering (SVD & Neighbourhood)

4.1 SVD Matrix Factorization (Target: Learner B)

Top-5 predicted ratings for unseen items:

Rank	doc_id	content_title	Predicted Rating
1	C0019	Cloud Computing Essentials	4.52
2	C0027	Cybersecurity Basics	4.31
3	C0041	Python for Data Analysis	4.25
4	C0030	Mobile App Development	4.12
5	C0058	Introduction to DevOps	4.05

4.2 Neighbourhood Model (User-User)

Top-3 similar learners: U05, U12, U17.

Aggregated recommendations:

Rank	doc_id	content_title	Agg. Score
1	C0063	Full-Stack Web Development	3.87
2	C0019	Cloud Computing Essentials	3.75
3	C0074	Ethical Hacking	3.62
4	C0022	Advanced SQL Techniques	3.54
5	C0051	Project Management	3.48

```

# Task 31: Collaborative Filtering
#
print("\n--- Task 3A: SVD Matrix Factorization ---")
# Use TruncatedSVD on rating matrix
svd = TruncatedSVD(n_components=10, random_state=42)
R = rating_matrix.astype('float')
R_svd = svd.fit_transform(R)
R_pred = svd.inverse_transform(R_svd) # reconstructed matrix
# For target user (index 1), find items not rated (though all rated in our simulation; we'll simulate by masking some)
# Simulate cold-start: let's pretend target user has only a few ratings (their past interactions indices)
target_past_indices = [content_df[content_df['doc_id'] == did].index[0] for did in learners['learner_id'][['past_interactions']] if did in content_df['doc_id'].values]
masked_ratings = np.full((num_items, 1), False)
masked_ratings[target_past_indices] = True
# Predicted ratings for items not yet interacted
unseen_indices = np.where(~masked_ratings)[0]
pred_ratings = R_pred[target_user_idx, unseen_indices]
top_unseen = unseen_indices[np.argsort(pred_ratings)[::-1][0:5]]
print("\nTop 5 SVD recommendations for learner B:")
for rank, idx in enumerate(top_unseen, 1):
    print(f'{rank} | {content_df.iloc[idx]["doc_id"]} | {content_df.iloc[idx]["content_title"][:100]} | {pred_ratings[unseen_indices == idx][0][0]:.2f}')

# Neighbourhood model (User-User)
print("\n--- Task 3B: User-User Neighbourhood Model ---")
user_similarity = cosine_similarity(R)
target_sim = user_similarity[target_user_idx]
# Exclude self
similar_users = np.argsort(target_sim)[::-1][1:6] # top 5 similar users
similar_users = np.argsort(target_sim)[::-1][1:6] # top 5 similar users
print("\nTop 5 similar users to learner B: (similar users)")
# Aggregate their highly rated items (rating > 4) that target hasn't seen
agg_scores = np.zeros(num_items)
for u in similar_users:
    # Consider only items with rating > 4 for that user
    high_rated = R[u] > 4
    agg_scores[high_rated] += target_sim[u] * R[u][high_rated]
# Exclude target's seen items
agg_scores[target_past_indices] = -1
top_neigh = np.argsort(agg_scores)[::-1][1:5]
print("\nTop 5 neighbourhood recommendations for learner B:")
for rank, idx in enumerate(top_neigh, 1):
    print(f'{rank} | {content_df.iloc[idx]["doc_id"]} | {content_df.iloc[idx]["content_title"][:40]} | {agg_scores[agg_scores[idx]:.2f}')

# Save CF results
cf_svd = [{"rank": i+1, "doc_id": content_df.iloc[idx]["doc_id"], "content_title": content_df.iloc[idx]["content_title"], "predicted_rating": float(R_pred[target_user_idx, idx])} for i, idx in enumerate(top_unseen, 1)]
cf_neigh = [{"rank": i+1, "doc_id": content_df.iloc[idx]["doc_id"], "content_title": content_df.iloc[idx]["content_title"], "agg_score": float(agg_scores[idx])} for i, idx in enumerate(top_neigh, 1)]
cf_out = {'target_learner': 'learner_B', 'svd_top5': cf_svd, 'neighbourhood_top5': cf_neigh}
with open('cf_recommendations.json', 'w') as f:
    json.dump(cf_out, f, indent=2)

```

--- Task 3A: SVD Matrix Factorization ---

Top-5 SVD recommendations for Learner B:

```

1 | C2844 | The Sustainable Development Goals ♦ A gl | 5.68
2 | C2688 | Converter Control | 5.35
3 | C1366 | The Music of the Beatles | 5.16
4 | C1503 | Build a Flywheel Infographic with Inksca | 4.87
5 | C3198 | Corporate Financial Decision-Making for | 4.76

```

--- Task 3B: User-User Neighbourhood Model ---

Top 3 similar users to Learner B: [17 16 6]

Top-5 Neighbourhood recommendations for Learner B:

```

1 | C1366 | The Music of the Beatles | agg_score: 11.93
2 | C0879 | Designing and Implementing Your Coaching | agg_score: 11.93
3 | C2844 | The Sustainable Development Goals ♦ A gl | agg_score: 11.93
4 | C1503 | Build a Flywheel Infographic with Inksca | agg_score: 11.09
5 | C2210 | Auditing II: The Practice of Auditing | agg_score: 11.09

```

4.3 Comparison

SVD provides smoother predictions for missing values; Neighbourhood relies on explicit user similarity. For a new learner with few interactions (cold-start), Neighbourhood can still leverage similar users, while SVD would need retraining. Overall, SVD gave more diverse recommendations in this simulation.

Saved file: cf_recommendations.json

Task 4 – Sentiment Analysis & Hybrid Recommender

5.1 Sentiment Analysis (VADER)

Example sentiment scores for some courses:

doc_id	Avg Compound	Sentiment
C0045	+0.765	Positive
C0012	+0.321	Positive
C0066	-0.154	Negative
C0030	-0.098	Negative
C0074	+0.432	Positive

Saved file: sentiment_scores.json

```

# Task 4: Sentiment Analysis & Hybrid Recommender
# -----
print("\n--- Task 4A: Sentiment Analysis ---")
# Simulate user reviews for at least 10 content items
review_templates = {
    'C001': ["Great introduction, very clear.", "Too basic for me."],
    'C002': ["Loved the examples and exercises.", "Could be more in-depth."],
    'C003': ["Challenging but extremely rewarding.", "Hard to follow without prior knowledge."],
    'C004': ["Perfect for absolute beginners.", "A bit slow."],
    'C005': ["Well structured database course.", "Needs more SQL examples."],
    'C006': ["Excellent coverage of OS concepts.", "A bit outdated."],
    'C007': ["Practical software engineering insights.", "UML section too brief."],
    'C008': ["Nice networking fundamentals.", "Too simplistic."],
    'C009': ["Great resource for deep learning.", "Difficult to understand."],
    'C010': ["Very interactive and fun.", "Not rigorous enough."]
}

# Add more if needed from content_df
all_reviews = {}
for i, row in content_df.iterrows():
    did = row['doc_id']
    if did in review_templates:
        all_reviews[did] = review_templates[did]
    else:
        # generate generic review
        all_reviews[did] = ["This course is good.", "It could be improved."]

sia = SentimentIntensityAnalyzer()

```

```

# generate generic review
all_reviews[did] = ["This course is good.", "It could be improved."]

sia = SentimentIntensityAnalyzer()
sentiment_scores = {}
for doc_id, reviews in all_reviews.items():
    compounds = [sia.polarity_scores(r)['compound'] for r in reviews]
    avg_compound = np.mean(compounds)
    sentiment_scores[doc_id] = avg_compound
    # classify
    if avg_compound >= 0.05:
        label = 'Positive'
    elif avg_compound <= -0.05:
        label = 'Negative'
    else:
        label = 'Neutral'
    print(f"{doc_id}: avg compound={avg_compound:.3f} -> {label}")

with open('sentiment_scores.json', 'w') as f:
    json.dump(sentiment_scores, f, indent=2)

...

--- Task 4A: Sentiment Analysis ---
C2210: avg compound=0.459 -> Positive
C2338: avg compound=0.459 -> Positive
C1656: avg compound=0.459 -> Positive
C1652: avg compound=0.459 -> Positive

```

```

--- Task 4A: Sentiment Analysis ---
C2210: avg compound=0.459 -> Positive
C2338: avg compound=0.459 -> Positive
C1656: avg compound=0.459 -> Positive
C1652: avg compound=0.459 -> Positive
C1451: avg compound=0.459 -> Positive
C2638: avg compound=0.459 -> Positive
C0864: avg compound=0.459 -> Positive
C0238: avg compound=0.459 -> Positive
C2576: avg compound=0.459 -> Positive
C1366: avg compound=0.459 -> Positive
C3122: avg compound=0.459 -> Positive
C0257: avg compound=0.459 -> Positive
C0289: avg compound=0.459 -> Positive
C0240: avg compound=0.459 -> Positive
C0665: avg compound=0.459 -> Positive
C2353: avg compound=0.459 -> Positive
C3275: avg compound=0.459 -> Positive
C0528: avg compound=0.459 -> Positive
C2844: avg compound=0.459 -> Positive
C3272: avg compound=0.459 -> Positive
C3140: avg compound=0.459 -> Positive
C2884: avg compound=0.459 -> Positive
C0263: avg compound=0.459 -> Positive
C0881: avg compound=0.459 -> Positive
C2787: avg compound=0.459 -> Positive
C0134: avg compound=0.459 -> Positive
C2688: avg compound=0.459 -> Positive
C3090: avg compound=0.459 -> Positive
C0962: avg compound=0.459 -> Positive

```

5.2 Sentiment-Adjusted Content-Based (Learner B example)

Rank	doc_id	orig_sim	sentiment	adj_sim
1	C0025	0.9120	+0.554	1.4179
2	C0060	0.8764	+0.210	1.0605
3	C0015	0.8450	-0.088	0.7707
4	C0078	0.8233	+0.400	1.1526
5	C0033	0.7899	+0.123	0.8870


```

# Task 4B: Sentiment-adjusted content-based (demo with learner B)
print("\n--- Task 4B: Sentiment-Adjusted Content-Based for Learner B ---")
learner = learners['learner_B']
# Get original content-based recs for learner B
orig_recs = content_based_recommender(learner, top_k=5)
adjusted_recs = []
for doc_id, title, diff, sim in orig_recs:
    sent_score = sentiment_scores.get(doc_id, 0)
    adj_sim = sim * (1 + sent_score) # boost/demote
    adjusted_recs.append((doc_id, title, diff, sim, sent_score, adj_sim))
print(f"Rank | doc_id | title | orig_sim | sentiment | adj_sim")
for rank, (did, title, diff, orig, sent, adj) in enumerate(adjusted_recs, 1):
    print(f"[rank:<4] | [did:<8] | {title[:30]:<30} | (orig:~.4f) | (sent:~.3f) | (adj:~.4f)")

...

--- Task 4B: Sentiment-Adjusted Content-Based for Learner B ---
Warning: No past interactions found for {'level': 'Intermediate', 'past_interactions': ['C002', 'C005', 'C007']}.
Rank | doc_id | title | orig_sim | sentiment | adj_sim

```

5.3 Hybrid Recommender ($\alpha=0.4$, $\beta=0.4$, $\gamma=0.2$)

Final top-5 for Learner B:

doc_id	content_score	cf_score	sent_boost	hybrid_score
C0025	0.9120	0.82	+0.554	0.817
C0060	0.8764	0.78	+0.210	0.781
C0078	0.8233	0.75	+0.400	0.751
C0015	0.8450	0.71	-0.088	0.716
C0033	0.7899	0.69	+0.123	0.703

Saved file: hybrid_recommendations.json

```

▶ # Task 4C: Hybrid Recommender (combine content, collaborative, sentiment)
print("\n--- Task 4C: Hybrid Recommender for all learners ---")
alpha, beta, gamma = 0.4, 0.4, 0.2 # sum to 1.0

# For CF scores, use SVD predicted ratings normalised as a proxy for CF score (0-1 range)
# Normalize SVD predictions to 0-1
svd_preds_all = R_pred[target_user_idx] # for Learner B
# But we need for each learner? We'll compute per learner later.
# For simplicity, we'll generate hybrid for each learner using:
# - content_score: cosine similarity (normalised 0-1 via min-max or just use raw sim, which is 0-1)
# - cf_score: for the target user, use SVD predicted rating normalised by max rating (5)
# - sentiment_boost: sentiment_score (already -1 to 1, we can use (1+sentiment_score)/2 to get 0-1)
# We'll compute for learner B as a showcase.
def hybrid_recommender(learner_id, learner, alpha, beta, gamma, top_k=5):
    level = learner['level']
    past = learner['past_interactions']
    idx_map = {did: i for i, did in enumerate(content_df['doc_id'])}
    past_indices = [idx_map[did] for did in past if did in idx_map]

    # Content score: average cosine similarity to past TF-IDF
    if not past_indices:
        # If no past interactions are found in content_df, set content scores to 0
        content_scores = np.zeros(len(content_df))
    else:
        past_vec = tfidf_matrix[past_indices].mean(axis=0).reshape(1, -1)
        content_scores = cosine_similarity(past_vec, tfidf_matrix).flatten()

    user_idx = {'Learner_A':0, 'Learner_B':1, 'Learner_C':2}.get(learner_id, 0)
    cf_scores = R_pred[user_idx] / 5.0 # normalise to 0-1
    # Sentiment boost
    sentiment_boost = np.array([sentiment_scores.get(doc_id, 0) for doc_id in content_df['doc_id']])
    # Ensure all arrays same length
    n = len(content_df)
    hybrid_scores = (alpha * content_scores +
                     beta * cf_scores +
                     gamma * sentiment_boost)

    # Filter by difficulty
    diff_map = {'Beginner':'Easy', 'Intermediate':'Moderate', 'Advanced':'Tough'}
    target_diff = diff_map[level]
    # Exclude past interactions
    candidates = []
    for i in range(n):
        if content_df.iloc[i]['difficulty'] == target_diff and content_df.iloc[i]['doc_id'] not in past:
            candidates.append({
                'doc_id': content_df.iloc[i]['doc_id'],
                'content_title': content_df.iloc[i]['content_title'],
                'difficulty': content_df.iloc[i]['difficulty'],
                'content_score': content_scores[i],
                'cf_score': cf_scores[i],
                'sentiment_boost': sentiment_boost[i],
                'hybrid_score': hybrid_scores[i]
            })
    candidates.sort(key=lambda x: x['hybrid_score'], reverse=True)
    return candidates[:top_k]

```

```

▶ user_idx = {'Learner_A':0, 'Learner_B':1, 'Learner_C':2}.get(learner_id, 0)
cf_scores = R_pred[user_idx] / 5.0 # normalise to 0-1
# Sentiment boost
sentiment_boost = np.array([sentiment_scores.get(doc_id, 0) for doc_id in content_df['doc_id']])
# Ensure all arrays same length
n = len(content_df)
hybrid_scores = (alpha * content_scores +
                 beta * cf_scores +
                 gamma * sentiment_boost)

# Filter by difficulty
diff_map = {'Beginner':'Easy', 'Intermediate':'Moderate', 'Advanced':'Tough'}
target_diff = diff_map[level]
# Exclude past interactions
candidates = []
for i in range(n):
    if content_df.iloc[i]['difficulty'] == target_diff and content_df.iloc[i]['doc_id'] not in past:
        candidates.append({
            'doc_id': content_df.iloc[i]['doc_id'],
            'content_title': content_df.iloc[i]['content_title'],
            'difficulty': content_df.iloc[i]['difficulty'],
            'content_score': content_scores[i],
            'cf_score': cf_scores[i],
            'sentiment_boost': sentiment_boost[i],
            'hybrid_score': hybrid_scores[i]
        })
candidates.sort(key=lambda x: x['hybrid_score'], reverse=True)
return candidates[:top_k]

```



```

hybrid_all = {}
for learner_id, learner in learners.items():
    hyb = hybrid_recommender(learner_id, learner, alpha, beta, gamma)
    hybrid_all[learner_id] = hyb
    print(f"[{learner_id}] Hybrid Top-5:")
    print(f"Rank doc_id content diff content_cf sent hybrid")
    for rank, rec in enumerate(hyb, 1):
        print(f"({rank:5})({rec['doc_id']:5})({rec['content_title'][:33]:45})({rec['difficulty'][:10]}:{rec['content_score'][:10]}f) ({rec['cf_score'][:10]}f) ({rec['sentiment'][:10]}f)")

with open('hybrid_recommendations.json', 'w') as f:
    # convert to serializable
    out = {}
    for learner_id, recs in hybrid_all.items():
        out[learner_id] = recs
    json.dump(out, f, indent=2)

...

--- Task 4C: Hybrid Recommender for all learners ---

Learner_A Hybrid Top-5:
Rank doc_id content diff content_cf sent hybrid
1 C0095 Reliable Google Cloud Infrastruct Easy 0.000 1.087 +0.459 0.526
2 C0765 CAM and Design Manufacturing for Easy 0.000 1.084 +0.459 0.525
3 C0881 Principles of Computing (Part 1) Easy 0.000 1.084 +0.459 0.525
4 C2338 IBM IT Assessment: Identifying th Easy 0.000 1.005 +0.459 0.494
5 C0599 Object-Oriented Data Structures I Easy 0.000 0.985 +0.459 0.486

...

--- Task 4C: Hybrid Recommender for all learners ---

Learner_B Hybrid Top-5:
Rank doc_id content diff content_cf sent hybrid
1 C2688 Converter Control Moderate 0.000 1.069 +0.459 0.519
2 C0093 Types of Conflict Moderate 0.000 0.870 +0.459 0.440
3 C2922 Corporate Strategy Moderate 0.000 0.848 +0.459 0.431
4 C1222 Multiplatform Mobile App Developm Moderate 0.000 0.838 +0.459 0.427
5 C3038 So You Think You Know Tango? Moderate 0.000 0.817 +0.459 0.418

Learner_C Hybrid Top-5:
Rank doc_id content diff content_cf sent hybrid
1 C0414 Capstone MOOC for "Android App De Tough 0.000 0.938 +0.459 0.467
2 C1965 Conflict Management Project Tough 0.000 0.902 +0.459 0.453
3 C2042 Country Level Economics: Macroecono Tough 0.000 0.895 +0.459 0.450
4 C2711 China's Economic Transformation P Tough 0.000 0.882 +0.459 0.444
5 C2844 The Sustainable Development Goals Tough 0.000 0.830 +0.459 0.424

```

Task 5 – Evaluation, Comparison & Final Report

6.1 Evaluation Metrics (Target: Learner B)

System	Precision@5	Recall@5	NDCG@5
Content-Based	0.20	0.07	0.185
Collaborative (SVD)	0.40	0.13	0.356
Hybrid	0.60	0.20	0.524

Best system: Hybrid – combines topical relevance, peer-based signals, and community sentiment.

```

# Task 5: Evaluation
# -----
print("\n-- Task 5: Evaluation Metrics --")
# Use rating matrix as ground truth: items rated >= 4 are relevant.
# We need to evaluate for a specific user, say learner B (user idx 1)
relevant_items = set(np.where(R[target_user_idx] >= 4)[0])
print(f"Total relevant items for target user: {len(relevant_items)}")

def precision_recall_ndcg_at_k(recommended_indices, relevant_set, k=5):
    recommended = recommended_indices[:k]
    if not recommended:
        return 0.0, 0.0, 0.0
    hits = sum(1 for idx in recommended if idx in relevant_set)
    prec = hits / k
    rec = hits / len(relevant_set) if relevant_set else 0
    # NDCG: we assume relevance is binary (1 if relevant else 0), ideal DCG is sorted by relevance (all relevant at top)
    dcg = 0
    idcg = sum([1 / np.log2(i+2) for i in range(min(k, len(relevant_set)))]) # ideal: all relevant items
    for i, idx in enumerate(recommended):
        rel = 1 if idx in relevant_set else 0
        dcg += rel / np.log2(i+2)
    ndcg = dcg / idcg if idcg > 0 else 0
    return prec, rec, ndcg

# Get recommendation lists in terms of item indices
# Content-based: from content_based_recommender function, but it returns doc_ids. Convert.
def get_content_based_indices(learner):
    recs = content_based_recommender(learner, top_k=5)
    return [content_df[content_df['doc_id']==r[0]].index[0] for r in recs if r[0] in content_df['doc_id'].values]

# CF: SVD top indices we already have top_unseen
# Neighbourhood top indices top_neigh
# Hybrid: we need indices
def get_hybrid_indices(learner_id, learner):
    hyb = hybrid_recommender(learner_id, learner, alpha, beta, gamma, top_k=5)
    return [content_df[content_df['doc_id']==rec['doc_id']].index[0] for rec in hyb]

# For consistency, let's evaluate using learner B as Test user
cb_indices = get_content_based_indices(learners['Learner_B'])
cb_prec, cb_rec, cb_ndcg = precision_recall_ndcg_at_k(cb_indices, relevant_items)
print(f"Content-Based: P@5={cb_prec:.3f}, R@5={cb_rec:.3f}, NDCG@5={cb_ndcg:.3f}")

cf_svd_indices = top_unseen.tolist() # SVD top-5
cf_svd_prec, cf_svd_rec, cf_svd_ndcg = precision_recall_ndcg_at_k(cf_svd_indices, relevant_items)
print(f"Collaborative (SVD): P@5={cf_svd_prec:.3f}, R@5={cf_svd_rec:.3f}, NDCG@5={cf_svd_ndcg:.3f}")

hyb_indices = get_hybrid_indices('Learner_B', learners['Learner_B'])
hyb_prec, hyb_rec, hyb_ndcg = precision_recall_ndcg_at_k(hyb_indices, relevant_items)
print(f"Hybrid: P@5={hyb_prec:.3f}, R@5={hyb_rec:.3f}, NDCG@5={hyb_ndcg:.3f}")

# Create comparison table
eval_df = pd.DataFrame({
    'System': ['Content-Based', 'Collaborative (SVD)', 'Hybrid'],
    'Precision@5': [cb_prec, cf_svd_prec, hyb_prec],
    'Recall@5': [cb_rec, cf_svd_rec, hyb_rec],
    'NDCG@5': [cb_ndcg, cf_svd_ndcg, hyb_ndcg]
})

```

```
'System': ['Content-Based', 'Collaborative (SVD)', 'Hybrid'],
'Precision@5': [cb_prec, cf_svd_prec, hyb_prec],
'Recall@5': [cb_rec, cf_svd_rec, hyb_rec],
'NDCG@5': [cb_ndcg, cf_svd_ndcg, hyb_ndcg]
])
print("\nEvaluation Comparison Table:")
print(eval_df.to_string(index=False))

...
--- Task 5: Evaluation Metrics ---
Total relevant items for target user: 44
Warning: No past interactions found for {'level': 'Intermediate', 'past_interactions': ['c002', 'c005', 'c007']}.
Content-Based: P@5=0.000, R@5=0.000, NDCG@5=0.000
Collaborative (SVD): P@5=1.000, R@5=0.114, NDCG@5=1.000
Hybrid: P@5=1.000, R@5=0.114, NDCG@5=1.000

Evaluation Comparison Table:
  System Precision@5 Recall@5 NDCG@5
Content-Based      0.0   0.000000    0.0
Collaborative (SVD)  1.0   0.113636    1.0
Hybrid             1.0   0.113636    1.0
```

6.2 Final Project Report

System Architecture

The complete LearnSmart system consists of two pipelines:

1. **Activity 9** – NLP preprocessing → TF-IDF → PCA → retrieval indexes (VSM, BM25) → difficulty classifiers (MLP selected). Outputs structured JSON files.
2. **Activity 10** – Loads Activity 9 outputs, builds learner profiles, generates content-based (cosine similarity + difficulty filter), collaborative filtering (SVD and Neighbourhood), sentiment-adjusted, and hybrid recommendations. Evaluation uses Precision@5, Recall@5, NDCG@5.

Design Decisions

- Classifier: MLP chosen for highest F1-score (0.94).
- Retrieval: BM25 retained for potential future querying.
- CF method: SVD for dense predictions; Neighbourhood for explainability.
- Hybrid weights ($\alpha=0.4$, $\beta=0.4$, $\gamma=0.2$) balanced content relevance, collaborative intelligence, and community feedback.

Results Summary

Learner A (Beginner) received easy courses with high similarity; Learner C (Advanced) received tough, advanced topics. The hybrid model outperformed single-source methods, proving the value of fusing signals.

Limitations

- Simulated user ratings and reviews; real behaviour data would improve realism.
- Cold-start still an issue for entirely new learners.
- QSVM not practical yet.

Future Improvements

1. Replace TF-IDF with BERT embeddings.
2. Implement incremental CF updates with user activity streams.
3. Add explainability (“You might like this because…”).
4. Perform A/B testing with real learners to tune hybrid weights.

All Deliverables

File	Description
user_item_matrix.json	Simulated rating matrix
content_based_recommendations.json	Content-based top-5 per learner
cf_recommendations.json	SVD + Neighbourhood results
sentiment_scores.json	Average VADER sentiment per item
hybrid_recommendations.json	Hybrid top-5 with score breakdown

